



Play & Eat

Integrantes del Grupo: Bessy Matamoros, Thayron Saldanha y Beatriz Palomo.

Curso académico: 2ºDAM

Tutor/Tutora del proyecto: Víctor Colomo

ÍNDICE PÁGINADO

Contenido

APARTADO 1) RESUMEN	2
APARTADO 2) JUSTIFICACIÓN DEL PROYECTO	2
APARTADO 3) OBJETIVOS	4
3.1) OBJETIVO GENERAL	4
3.2) OBJETIVOS ESPECÍFICOS	5
APARTADO 4) DESARROLLO	5
4.1) FUNDAMENTACIÓN TEÓRICA	5
4.2) MATERIALES Y MÉTODOS.....	6
4.3) RESULTADOS Y ANÁLISIS	10
APARTADO 5) CONCLUSIONES	11
APARTADO 6) LÍNEAS DE INVESTIGACIÓN FUTURAS.....	11
APARTADO 7) BIBLIOGRAFÍA Y WEBGRAFÍA	12

APARTADO 1) RESUMEN

Este proyecto consiste en el desarrollo de una plataforma web denominada Play & Eat. Su objetivo principal es facilitar a las familias la búsqueda de restaurantes que cuenten con servicios específicos para niños, como zonas de juego, menús infantiles o animación, permitiendo así que tanto adultos como pequeños disfruten de su tiempo de ocio.

La aplicación resuelve el problema de la dispersión de información y la falta de filtros específicos en las plataformas gastronómicas actuales. A través de una interfaz intuitiva, los usuarios pueden encontrar locales adaptados a sus necesidades, mientras que los dueños de negocios disponen de una herramienta para promocionar sus instalaciones ante un público objetivo muy concreto. Los resultados esperados incluyen una mejora en la planificación familiar y un fomento del ocio compartido de calidad.

APARTADO 2) JUSTIFICACIÓN DEL PROYECTO

La idea de este proyecto nace de una problemática real detectada en el día a día de las familias con niños pequeños. Existe una gran dispersión de información a la hora de organizar planes de ocio gastronómico. Aunque los creadores de contenido muestran sitios ideales en redes sociales, esa información termina perdida en guardados de Instagram o TikTok que resultan imposibles de localizar cuando realmente se necesitan.

El problema principal que se pretende resolver es la falta de un lugar centralizado para esos "planes de sobremesa". Las quedadas con amigos se vuelven complicadas porque, una vez terminada la comida, los niños entran en un estado de aburrimiento, por lo que muchas familias recurren a las pantallas para alargar la reunión.

Play & Eat surge para concentrar toda esa oferta dispersa en una sola herramienta, de forma que las familias tengan la seguridad de que, al elegir un sitio en la plataforma, los niños tendrán la oportunidad de jugar mientras los adultos disfrutan de la sobremesa, eliminando la dependencia de los dispositivos electrónicos.

Análisis Comparativo con aplicaciones existentes

Durante la fase de investigación, se constató la ausencia de aplicaciones web o móviles específicas dedicadas exclusivamente a la búsqueda técnica de restaurantes con servicios infantiles integrados. La oferta actual se encuentra fragmentada principalmente en dos tipos de alternativas:

1. **Blogs y portales de ocio familiar (ej. Sapos y Princesas, Time Out):** Son plataformas de contenido editorial y artículos recomendados. Aunque ofrecen sugerencias de calidad, la información es estática, no permite un filtrado técnico avanzado por mapa o servicios, y depende de que el usuario busque activamente entre artículos o listados preexistentes.
2. **Plataformas gastronómicas generalistas (ej. TripAdvisor, Yelp):** Son grandes bases de datos de restauración orientadas al público general. Aunque incluyen etiquetas amplias, carecen de filtros específicos para el detalle del entretenimiento infantil.

A continuación, se presenta un análisis comparativo que justifica la propuesta de valor de **Play & Eat** frente a las alternativas detectadas en el mercado:

Característica	Portales de Ocio (Sapos y Princesas / Time Out)	Plataformas Generales (TripAdvisor / Yelp)	Play & Eat
Formato de Información	Artículos, guías estáticas y blogs de recomendaciones.	Fichas de establecimientos y opiniones de usuarios.	Buscador interactivo y dinámico en tiempo real.
Filtro "Family Friendly"	Basado en la línea editorial del propio blog.	Genérico y subjetivo, basado en comentarios individuales.	Basado en servicios técnicos verificados por el negocio.
Detalle de servicios	Menciones generales dentro del texto del artículo.	No distingue entre tipos de entretenimiento.	Filtros específicos: Animación, Monitores, Zona de juegos.

Característica	Portales de Ocio (Sapos y Princesas / Time Out)	Plataformas Generales (TripAdvisor / Yelp)	Play & Eat
Uso de la Cartografía	Listados de direcciones sin mapa interactivo avanzado.	Geolocalización general orientada al turista.	Mapa interactivo optimizado para la búsqueda local inmediata.
Panel para Negocios	No disponible (el contenido lo redacta el portal).	Gestión de ficha genérica sin enfoque familiar.	Panel específico para publicar servicios orientados a familias.

La principal ventaja competitiva de Play & Eat radica en la centralización y especialización: mientras que los blogs ofrecen una lectura pasiva y las plataformas como TripAdvisor están diseñadas para el usuario general, este proyecto unifica lo mejor de ambos mundos. Convierte las recomendaciones dispersas de internet en una herramienta técnica y visual de búsqueda inmediata para las familias locales.

APARTADO 3) OBJETIVOS

3.1) OBJETIVO GENERAL

Desarrollar una plataforma integral (web y móvil-responsiva) que conecte a familias con restaurantes que ofrecen servicios de entretenimiento y cuidado infantil.

3.2) OBJETIVOS ESPECÍFICOS

- Implementar un sistema de autenticación seguro diferenciando entre perfiles de familia y de negocio.
- Crear una base de datos relacional y escalable para almacenar información detallada de los locales, incluyendo fotos.
- Desarrollar un buscador inteligente con filtros por categoría (Brunch, Comida, Merienda, Cena) y por servicios específicos de entretenimiento.
- Diseñar un panel de administración intuitivo para que los dueños de los restaurantes puedan gestionar su información de forma autónoma.
- Asegurar que la web sea totalmente responsiva, permitiendo que se use cómodamente desde el móvil en cualquier lugar (filosofía Mobile First).

APARTADO 4) DESARROLLO

4.1) FUNDAMENTACIÓN TEÓRICA

- **Next.js (Framework Fullstack):** Framework basado en React que se ha utilizado para construir la aplicación completa. Gestiona tanto el renderizado de la interfaz como la lógica del servidor mediante API Routes propias. Una de sus características más relevantes es el soporte para Server-Side Rendering (SSR), lo que permite que las páginas se generen en el servidor antes de enviarse al navegador, mejorando significativamente el rendimiento y la indexación en buscadores (SEO). *Justificación de su elección:* a diferencia de un proyecto React puro, Next.js permite centralizar frontend y backend en un único proyecto, reduciendo la complejidad y el tiempo de desarrollo.
- **Tailwind CSS:** Framework de estilos basado en clases de utilidad. En lugar de escribir hojas de estilo CSS separadas, los estilos se aplican directamente en el HTML/JSX mediante clases predefinidas. Su sistema de diseño permite construir interfaces responsivas y consistentes de forma muy rápida, y el resultado final tiene un peso de CSS mínimo al eliminar automáticamente los estilos no utilizados.
- **Supabase (Base de Datos):** Plataforma de base de datos como servicio (DBaaS) que proporciona una instancia de PostgreSQL en la nube con panel de administración visual. Sus características principales son la gestión de

autenticación de usuarios, el almacenamiento de archivos y las políticas de seguridad a nivel de fila (Row Level Security - RLS). Se ha elegido por su robustez, su integración nativa con sistemas de autenticación mediante JWT y la posibilidad de gestionar todos los datos desde un único panel sin necesidad de configurar un servidor propio.

4.2) MATERIALES Y MÉTODOS

Metodología de Trabajo y Organización del Equipo

Para el desarrollo de este proyecto, se ha seguido una metodología de trabajo flexible pero organizada, adaptada a la disponibilidad temporal del equipo de desarrollo. En lugar de un modelo rígido, se optó por un enfoque práctico basado en la comunicación constante y el aprendizaje iterativo. Los pilares de la organización han sido:

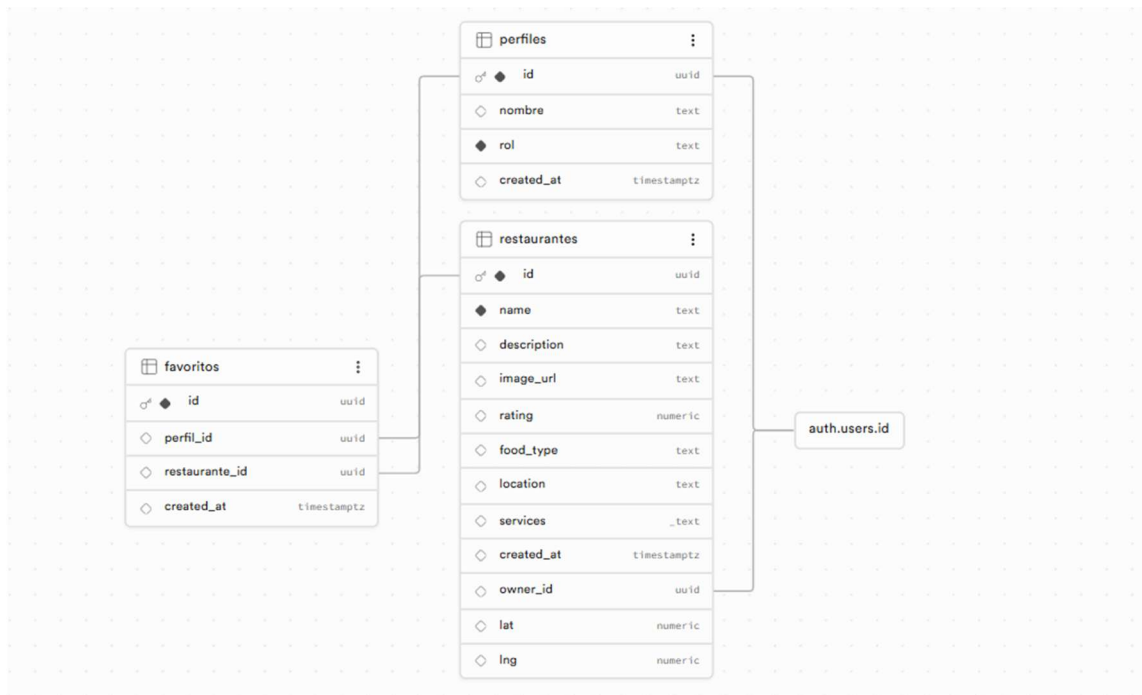
- **Gestión de tareas con Trello:** Al inicio del proyecto, se creó un tablero en Trello donde se volcaron todas las ideas y funcionalidades. Se dividió el trabajo en tarjetas (tareas pendientes, en proceso y finalizadas), lo que permitió visualizar el avance y realizar el reparto de los módulos básicos de la aplicación según las habilidades individuales de los integrantes.
- **Reuniones de sincronización:** Debido a la complejidad de los horarios compartidos, se estableció un sistema de reuniones semanales en horario nocturno. En estas sesiones se realizaba la revisión del trabajo autónomo, se resolvían los bloqueos técnicos detectados y se acordaban los pasos a seguir durante la semana siguiente.
- **Control de versiones con Git y GitHub:** El código fuente se ha gestionado de forma colaborativa mediante un repositorio en GitHub. Cada miembro del equipo publicaba sus aportaciones y las subía al repositorio remoto (*push*). El resto de integrantes descargaba las actualizaciones (*pull*) para asegurar que el trabajo se desarrollase sobre la versión más reciente del proyecto, asimilando de manera práctica la gestión de los conflictos de código surgidos en el proceso.

Planificación del Proyecto

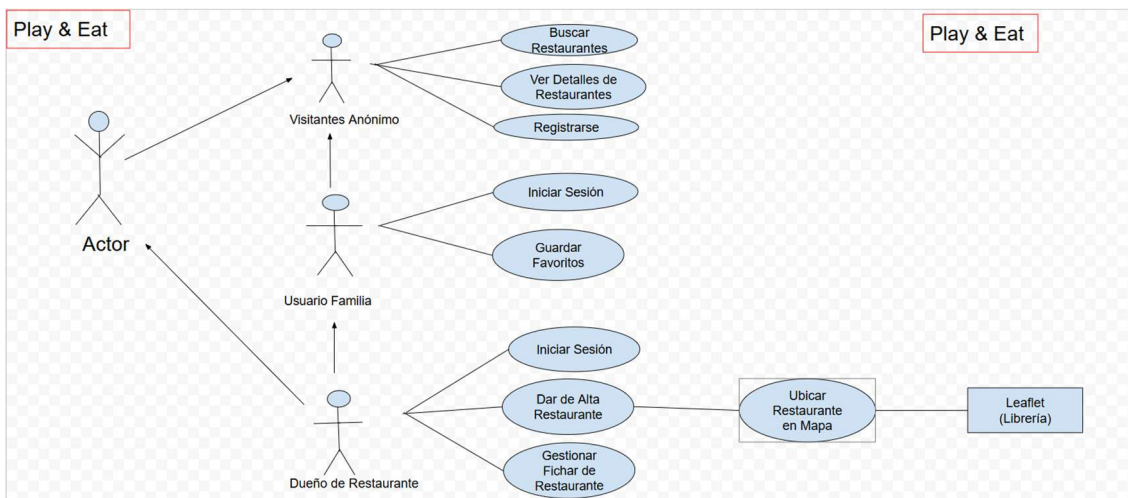
El proyecto se inició la última semana de enero. La planificación ha estado marcada por una fase inicial de ideación, un desarrollo progresivo de la interfaz y un período final intensivo de desarrollo de backend junto con la integración del sistema.

Fase	Descripción	Periodo
I. Ideación y Definición	Lluvia de ideas, selección del concepto y definición de funcionalidades principales.	Enero (S. 4) - Feb (S. 1)
II. Marca e Identidad	Diseño del logo y elección de colores y tipografía.	Febrero (Semana 2)
III. Desarrollo Frontend	Fase principal: creación de componentes (RestaurantCard, CategoryNav, HeroBanner, etc.) y todas las vistas de la interfaz.	Feb (S. 3) - Mayo (S. 1)
IV. Backend e Integración	Creación de la arquitectura de API (Rutas, Controladores y Servicios) para conectar de forma segura el frontend con Supabase.	Mayo (Semana 2)
V. Pulido y Detalles	Ajustes finales de estilos CSS, corrección de errores de última hora y estudio del mapa interactivo.	Mayo (Semana 3)

- **Diagrama de la base de datos:**



- **Diagramas de casos de uso:**



- **Estructura del proyecto:**

El proyecto sigue una estructura de carpetas organizada por responsabilidades, lo que facilita el mantenimiento y la escalabilidad:

- **/app:** Contiene todas las rutas y páginas de la aplicación, gestionadas por el sistema de enrutamiento *App Router* de Next.js. Dentro de esta carpeta se distinguen dos tipos de contenido: las páginas visibles para el usuario (ej. /restaurantes, /perfil, /admin) y los

puntos de entrada de la API REST (carpeta /app/api/), que actúan como el backend del sistema.

- **/components:** Almacena piezas de interfaz reutilizables e independientes, como RestaurantCard.tsx, Header.tsx o HeroBanner.tsx. Este patrón evita la duplicidad de código y garantiza que cualquier cambio visual se aplique de forma consistente en toda la aplicación.
- **/lib:** Contiene el núcleo lógico del backend, organizado en dos subcarpetas:
 - **/lib/backend/controllers/:** Los **controladores** (ej. authController.ts, restaurantController.ts) reciben las peticiones de la API, validan los datos y delegan la operación al servicio correspondiente.
 - **/lib/backend/services/:** Los **servicios** (ej. authService.ts, favoriteService.ts) son los únicos que interactúan directamente con la base de datos de Supabase, manteniendo la lógica de negocio separada del resto.
- **/public:** Almacena recursos estáticos como el logotipo e iconos de la aplicación.

Breve análisis del código

El código de la aplicación se estructura bajo un patrón de diseño modular que separa la interfaz gráfica de usuario (el frontend) de la lógica de negocio y gestión de datos (el backend). El objetivo de este análisis es detallar la secuencia que siguen los datos del proyecto a través de un escenario fundamental del sistema: el alta de un nuevo establecimiento gastronómico por parte de un usuario con rol de negocio.

1. Origen de los datos en la Capa de Cliente (Frontend)

El proceso comienza en el formulario de la página de administración (/app/admin/page.tsx). Cuando el usuario introduce la información obligatoria del restaurante (como el nombre, el tipo de comida, los servicios infantiles y las coordenadas geográficas) y confirma la acción, la interfaz captura estos datos. A través de una función asíncrona, el navegador realiza una petición web que empaqueta la información en formato JSON y la envía hacia la ruta interna del servidor correspondiente a /api/restaurantes.

2. Gestión y validación en la Capa de Control (Backend - Controller)

La solicitud es recibida por el enrutador de Next.js en el archivo `/app/api/restaurantes/route.ts`, que actúa como la puerta de acceso al servidor. Este archivo transfiere los datos inmediatamente al controlador específico (`restaurantController.ts`), ubicado en la carpeta `/lib/backend/controllers`. La función del controlador es actuar como un filtro de seguridad: analiza el contenido recibido y verifica que ningún campo obligatorio esté vacío. En caso de detectar anomalías o datos incompletos, el controlador interrumpe el proceso de forma inmediata y devuelve un mensaje de error a la pantalla, evitando procesar información incorrecta.

3. Persistencia en la Capa de Datos (Backend - Service)

Una vez superados los controles, el controlador invoca al servicio especializado `restaurantService.ts`, situado en la carpeta `/lib/backend/services`. Esta capa se dedica en exclusiva a la comunicación con el sistema de almacenamiento. Utilizando la configuración centralizada en `supabase.ts`, el servicio ejecuta una instrucción asíncrona para guardar de forma permanente el registro del nuevo restaurante en la tabla correspondiente de Supabase en la nube.

4. Cierre del ciclo y respuesta visual

Tras confirmar que el almacenamiento se ha realizado con éxito, el servidor devuelve un aviso de confirmación hacia el frontend de la aplicación. Al recibir esta respuesta, la interfaz de usuario se actualiza dinámicamente sin necesidad de recargar la página entera. El componente del mapa interactivo (`RestaurantsMap.tsx`) lee las nuevas coordenadas geográficas guardadas y dibuja al instante un marcador con su chincheta y su ventana emergente (*popup*) en la posición exacta del mapa.

Este análisis demuestra que el código se encuentra completamente desacoplado en tres niveles (pantalla, controlador y servicio). Esta separación garantiza que si en el futuro se requiriese modificar o sustituir la base de datos, únicamente se verían afectados los archivos de la capa de servicios, manteniendo intacto el diseño visual y el comportamiento de la interfaz de usuario.

4.3) RESULTADOS Y ANÁLISIS

Se ha logrado implementar una aplicación funcional que cumple con todos los requisitos iniciales. Las familias pueden registrarse y acceder a un listado dinámico de restaurantes, mientras que los dueños de negocios pueden publicar sus locales con éxito. El rendimiento del sistema es excelente gracias al uso de Next.js

APARTADO 5) CONCLUSIONES

El desarrollo de Play & Eat ha permitido aplicar de forma práctica los conocimientos adquiridos durante el grado, enfrentando retos reales de la ingeniería de software como la gestión de sesiones, el diseño de APIs REST seguras y el desarrollo de interfaces responsivas. Se ha cumplido el objetivo de crear una herramienta útil que resuelve un problema real de las familias actuales.

APARTADO 6) LÍNEAS DE INVESTIGACIÓN FUTURAS

Como ampliaciones identificadas para el futuro del proyecto, se proponen las siguientes líneas de desarrollo:

1. **Sistema de Recomendaciones basado en IA:** Implementar algoritmos de aprendizaje automático para sugerir restaurantes de forma personalizada según la edad de los hijos y el historial de visitas de la familia.
2. **Comunidad y Quedadas (Playdates):** Crear un espacio social donde las familias puedan contactar entre sí para organizar comidas grupales y fomentar la socialización de los niños.
3. **Realidad Aumentada (Tour Virtual):** Integrar visualizaciones en 3D o realidad aumentada de las zonas de juego para que los padres puedan evaluar la seguridad de las instalaciones antes de realizar la reserva.
4. **Sistema de valoraciones:** Permitir que las familias publiquen comentarios y valoraciones sobre los servicios infantiles de cada restaurante.
5. **Sistema de Reservas:** Permitir reservar mesa directamente desde la plataforma, indicando el número de niños y adultos.



APARTADO 7) BIBLIOGRAFÍA Y WEBGRAFÍA

Browse fonts. (n.d.). Google Fonts. Retrieved May 16, 2026, from <https://fonts.google.com/>

Installing with Vite - installation. (n.d.). Tailwindcss.com. Retrieved May 16, 2026, from <https://tailwindcss.com/docs>

Leaflet — an open-source JavaScript library for interactive maps. (n.d.). Leafletjs.com. Retrieved May 16, 2026, from <https://leafletjs.com>

Lucide icons. (n.d.). Lucide. Retrieved May 16, 2026, from <https://lucide.dev/>

midulive [@midulive]. (n.d.). *Tutorial Next.js 14 paso a paso, para Principiantes* [Video]. Youtube. Retrieved May 16, 2026, from <https://www.youtube.com/watch?v=jMy4pVZMyLM>

Next.js Docs. (n.d.). Nextjs.org. Retrieved May 16, 2026, from <https://nextjs.org/docs>

Supabase. (2026, March 18). *Supabase docs.* Supabase.com. <https://supabase.com/docs>

(N.d.). Stackoverflow.com. Retrieved May 16, 2026, from <https://stackoverflow.com/>